Systems Administration ASIST

Topic 3

Access management

Access management

- ***Network Access Control*** (**NAC**) is a concept of controlling access to an infrastructure by implementing security policies
- Its goals are to
  - Mitigate and/or prevent *zero-day* attacks
  - Apply a security policy across the infrastructure
  - Use identities to perform access control
- These objectives can be achieved through various mechanisms (security policies with definition of security control, filtering, prevention, among others)
- NAC will act as an automatic detection and response system that can react in real time to prevent threats before they do damage

Access management

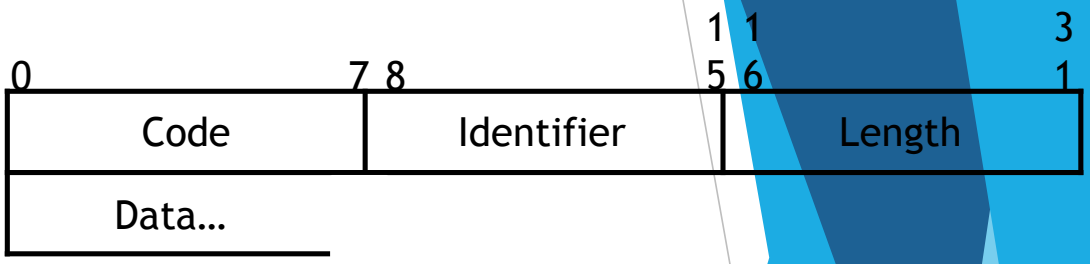
- NAC can be implemented using two criteria separately or a mixture of both
  - Pre-admission
    - Requires a system to meet all security requirements (such as installed updates, updated antivirus) before it can communicate with the network
  - Post-admission
    - Allow or deny access based on user activity, which in turn is based on a previously defined authorization matrix
- The *pre-admission* criterion refers to systems, services, and will be analyzed later
- But the *post-admission* criteria is clear that involves users who access the network
- And how is performed the identification of users?

Access management

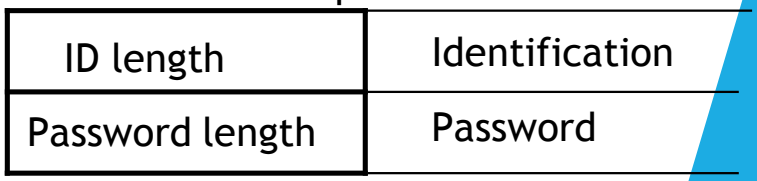
- Users represent the actors of an IT infrastructure
- Need infrastructure authentication to take advantage of your resources
- The simplest authentication is called ***Password Authentication Protocol*** (**PAP**) and consists of the exchange of two messages at the beginning of the establishment of authentication
  - User sends identification and password
  - The authentication system validates the identification / password pair
  - Returns authorization or denial of access

Access management

- This method is considered unsafe because
  - The identification and password are sent in clear text (readable)
  - There is no protection against repetitions or successive failures
  - The identification and password need to be previously defined (shared secret)
- The code represents the operation in progress
  - 1 – Authentication request
  - 2 – Authentication Ack
  - 3 – Authentication Nak

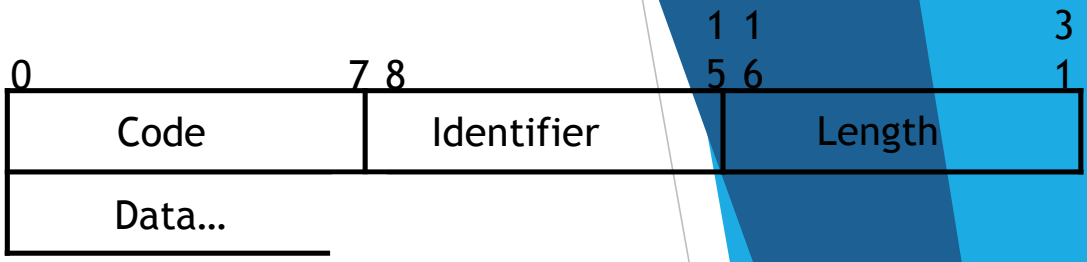


Data field in depth

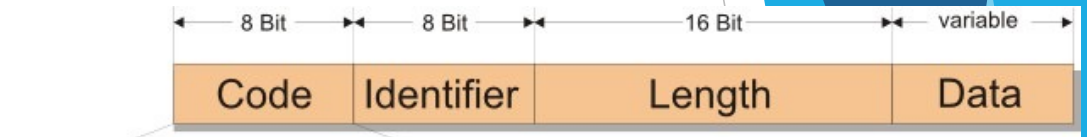


Access management

- An alternative and more secure method is applied by the ***Challenge-Handshake Authentication Protocol*** (**CHAP**)
- This protocol uses a three message system and can periodically verify the user's identity
- However, it is still necessary to define the identification and password in advance (shared secret)
- The Code field indicates the operation in progress
- The Identifier field is a random number and serves to map challenges and responses
- The Data field contains
  - The size of the challenge or response text
  - The challenge (if code = 1) or the answer (if code = 2)
  - A name that identifies the device that sent the challenge



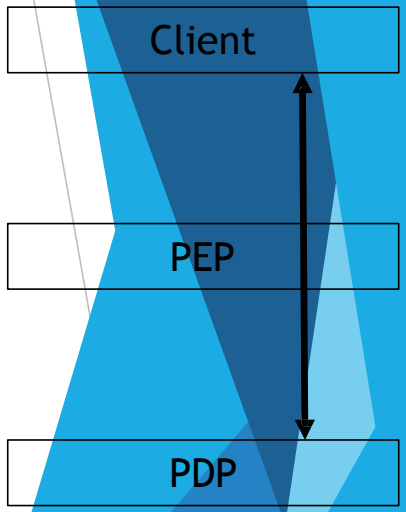
The challenge is a random number generated by the service and linked to the password
Varies from session to session



Coding (hex)	Meaning
01	Challenge
02	Response
03	Success
04	Failure

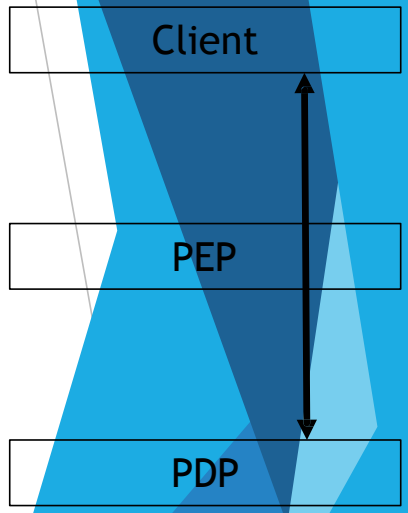
Access management

- The network access point is usually called the ***Policy Enforcement Point*** (**PEP**), which in turn:
  - Or contain user identification and credentials to validate access
  - Or contact a ***Policy Decision Point*** (**PDP**) who will be in charge of validating the entered credentials



Access management

- Although it is not specifically a protocol, it is also worth looking at the ***Extensible Authentication Protocol*** (**EAP**)
- Actually it is a framework for authentication
- Enables multiple alternative authentication systems, such as card use, biometrics, etc.
- Allows the network access point (***Policy Enforcement Point*** **PEP**) to work in pass-through mode, for example when the connection is made through a wireless access point
- In practice, it makes it possible for the PEP to be transparent and authentication to be directly at the PDP
- Several security mechanisms are possible to implement using EAP, of which the ***Protected Extensible Authentication Protocol*** (**PEAP**) stands out, which encapsulates EAP in a ***Transport Layer Security*** (**TLS**) tunnel
- ***Wifi Protected Access*** (**WPA**) standards include PEAP



Access management

- One of the most used authentication mechanisms is based on a shared secret (such as a password)
- The pre-shared secret is part of the symmetric cipher, which we will see in another topic
- The pre-shared secret guarantees confidentiality as it requires the key to decrypt the message
- Some authors consider that this method does not guarantee authenticity
  - Because the secret may be weak or obtained by social engineering (which we will return to in another topic)
- For a pre-shared secret to be effective, it must be exclusive to a pair of entities
  - That is, for N entities there must be $C_{(n,2)}$ (combinations of N two by two)

Access management

- The high number of secrets leads to human problems
  - How we generate and remember a huge number of secrets?
- It also implies the prior creation of the secret - for example, a user account
- In practice, this means that a previously unknown entity can never authenticate itself
- Also in a practical context, how do we share the secret with an entity? Sending by SMS?

Access management

- Authentication of entities is part of **Identity Management**
- Can be centralized or decentralized
  - Decentralized authentication (also known as distributed access control) implies that several entities located throughout the system or infrastructure perform access verification
    - Synchronization is complex and requires a high administrative effort
    - Users tend to use the same password for all services and systems, or write down passwords on paper, with direct implications for security
    - However it offers advantages in access authorization control
  - Centralized authentication requires less administrative effort, does not require synchronization of passwords, and entities need to authenticate only once
    - In short, it provides Single Sign-On (SSO)
    - But it requires an increased effort for access authorization

Access management

- In organizations, the need to ensure the centralized management of users is particularly felt
- Several of these services are based on *Lightweight Directory Access Protocol* (**LDAP**) RFC 3494
  - Microsoft's Active Directory Domain Services, for example, is based on LDAP
- LDAP can be considered an evolution of the X.500 protocol, supporting dialogue with it and keeping several of its definitions (such as classes and attributes)
- An important advantage of LDAP is that it supports exporting branches of the tree (or the entire tree, of course.!)

Access management

- LDAP (RFC 4511 in its latest version) is a non-relational database that can host entity authentication
- Its data structure is very flexible and optimized for query operations
- Its structure is in a tree, each branch being an **object** that has **attributes**
- The top of the structure is called **root**
- Attributes are indexed in order to optimize the search/query
- The client (human or system) accesses the global directory (which can contain multiple LDAP servers) through a client or ***Directory User Agent*** (**DUA**)
- This client in turn interacts with one or more servers or *Directory System Agents* (**DSA**) through the protocol LDAP

Access management

- The LDAP protocol uses TCP as a transport mechanism and is based on ***Abstract Syntax Notation One*** (**ASN.1**)
- All LDAP messages are encapsulated in a format called ***LDAPMessage***, a string in string format that contains a sequence number, an identifier, the intended operation, and a message
- If a message in which the sequence number is not recognized, the connection is terminated by returning a disconnection indication with an error code (**ProtocolError**) to the client
- In all other cases where it is not possible for the client or server to analyze the message, the connection is immediately terminated and may or may not return a message to the other end

Access management

- The definition of the attributes of objects in LDAP must comply with the definition of the *schema* (defined in RFC 4512) and is therefore not free
- The *schema* contains the possible classes each containing the attributes that are valid and what kind of use is allowed in each one
  - Some attributes are required, others are optional
- The definition of classes implements the concept of inheritance, so that when a class is a subclass of another, it incorporates the attributes of the hierarchically superior, and more attributes can be defined in the subclass

Access management

- There are structural and auxiliary classes (however optional)
- Each object belongs to a single structural class, but can additionally belong to several auxiliary classes
- Among the mandatory attributes is the **Domain Component** (**DC**) which can belong to the ***dcObject*** class (which is an auxiliary) usually combined with the structural class ***Organization***
  - Alternatively you can use the structural class **Domain**
- Naming based on the DNS domain name has become common, with each of its parts being a **dc**, separated by commas
  - DNS: **isep.ipp.pt**
  - root: **dc=isep,dc=ipp,dc=pt**

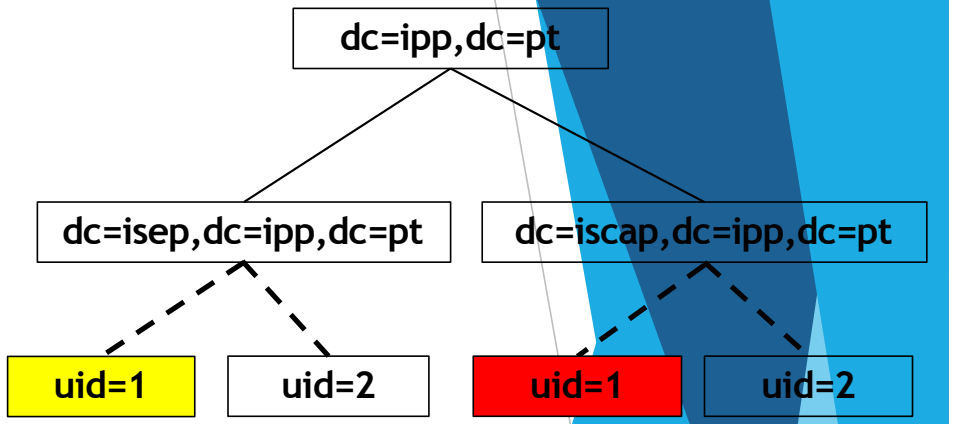
Access management

- The name that identifies a single, unique object in the LDAP tree is a ***Distinguished Name*** (**DN**)
- If the DN corresponds to ***root***, it is called **base DN** or **root DN**
- A DN must be unique in LDAP unambiguously identifying the object
- Attribute names however can be repeated throughout the structure, provided they are on different branches of the tree
- ***Relative Distinguished Names*** (**RDN**) are then distinguished, which, being unique and exclusive in the branch in which they are found, can be repeated in another branch.
- In the image, objects with RDN 1 have the DN

  (with yellow background) **uid=1,dc=isep,dc=ipp,dc=pt**

  (with red background) **uid=1,dc=iscap,dc=ipp,dc=pt**

Under dc=isep, dc=ipp, dc=pt there can only be one uid=1, being one RDN, yet there may be others uid <> 1, as under dc=iscap, dc=ipp, dc=pt

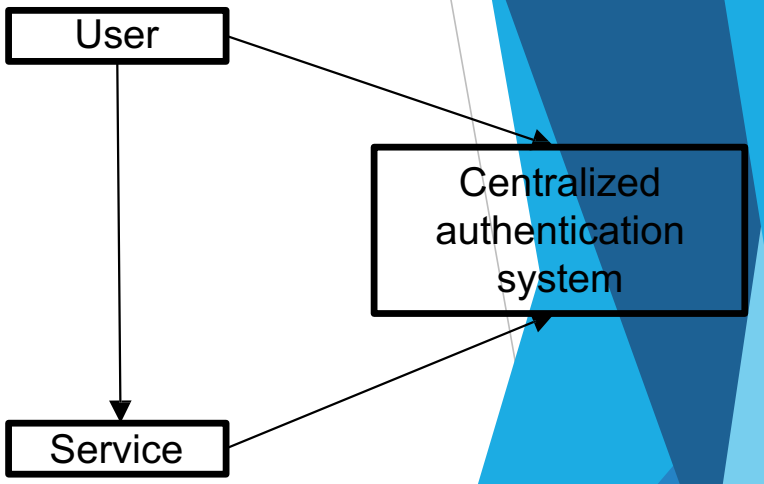


Access management

- When an entity authenticates to an infrastructure there is a sequence of necessary steps
- First, the **identification**
  - It consists of announcing who the entity is (for example, the username, which we will call ID, and the access password)
- Afterwards, the **authentication**
  - The system validates the ID – password pair
  - Returns identification success or failure
- These steps will have to be performed for all services that the ID wants to access
- But will have access to them (authorization)?

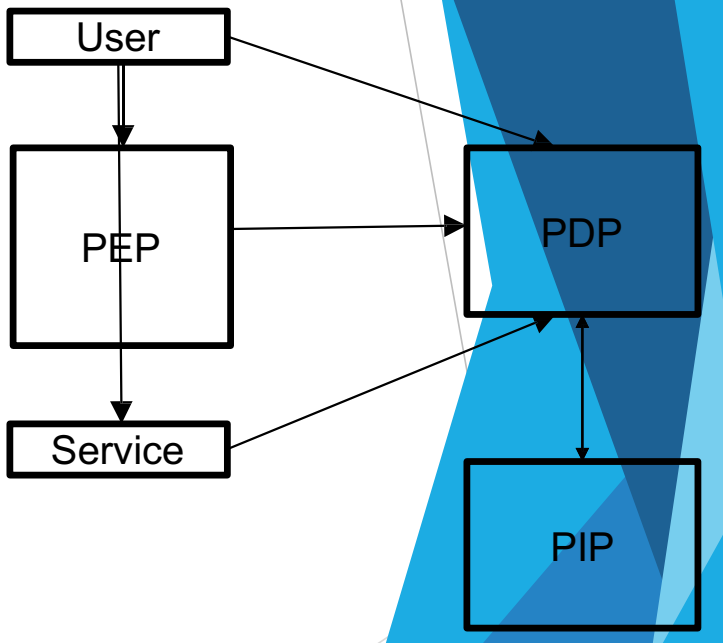
Access management

- **Authentication** refers to proving the identification of the person who authenticates
- **Authorization** refers to verifying that the ID can access the service
  - And, of course, it only occurs to those who have successfully authenticated
- The advantage of having a centralized authentication service is thus clear, as it allows all services to validate the user
- This does not imply, however, that the user has guaranteed authorization to all services; implies is that the service can validate the user using the centralized authentication service



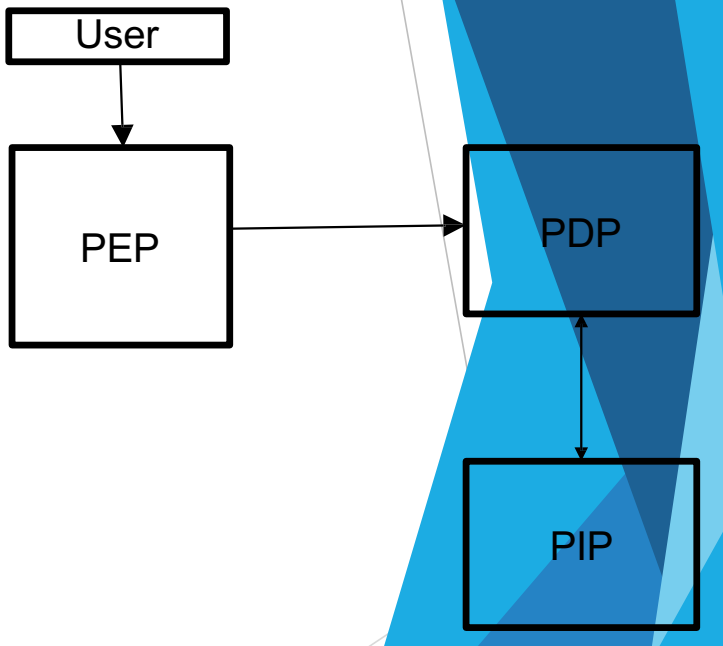
Access management

- The central authentication service is commonly referred to as ***Policy Decision Point*** (**PDP**)
- The validation of the authorization granted to the user uses another centralized service, closely linked to the PDP, called ***Policy Information Point*** (**PIP**)
- The user does not normally access the PDP directly but to a service that is responsible for contacting the PDP
- This service is called ***Policy Enforcement Point*** (**PEP**)
- PEP is normally a network service, but it can also be VPN tunnel access (which we'll get back to in another module) or an Access Point
  - In this case, it is usually called ***Network Access Service*** or ***Network Access Server*** (**NAS**)

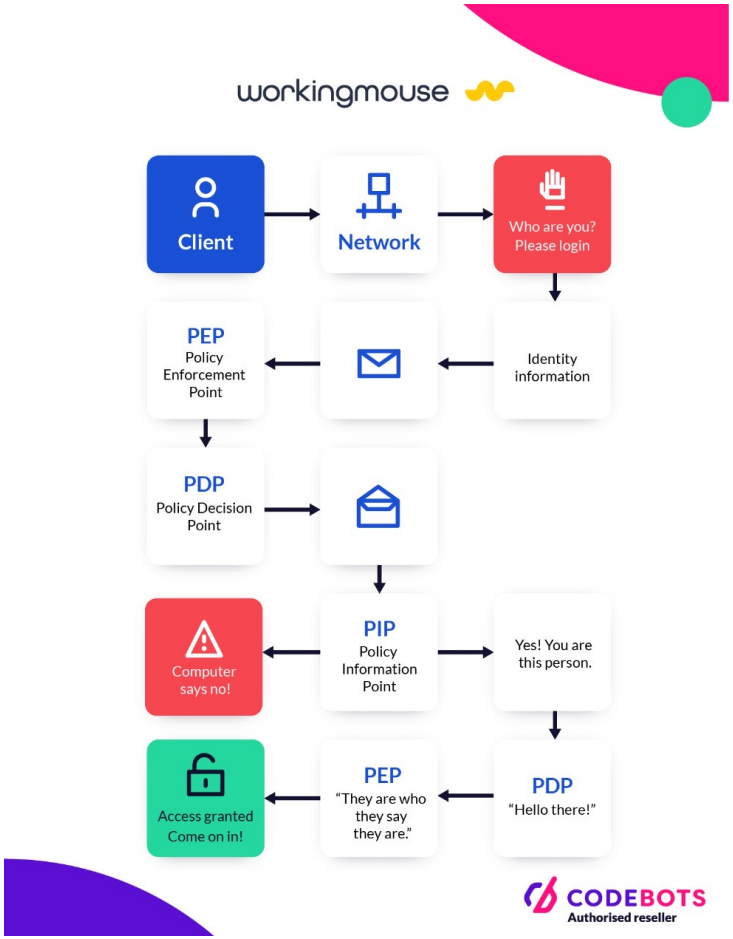


Access management

- The central authentication service is commonly referred to as ***Policy Decision Point*** (**PDP**)
- The validation of the authorization granted to the user uses another centralized service, closely linked to the PDP, called ***Policy Information Point*** (**PIP**)
- The user does not normally access the PDP directly but to a service that is responsible for contacting the PDP
- This service is called ***Policy Enforcement Point*** (**PEP**)
- PEP is normally a network service, but it can also be VPN tunnel access (which we'll get back to in another module) or an Access Point
  - In this case, it is usually called ***Network Access Service*** or ***Network Access Server*** (**NAS**)



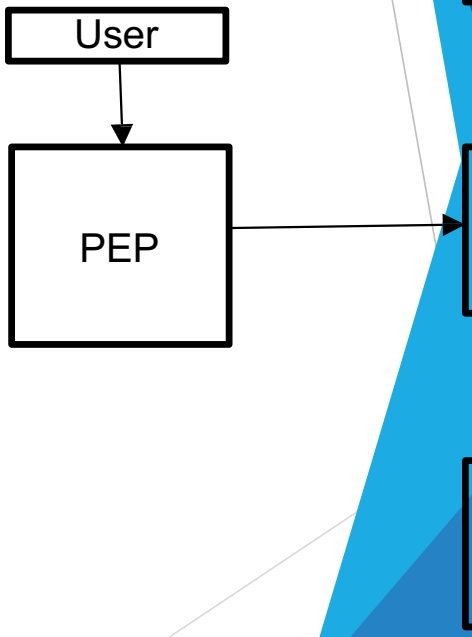
Access management



Source: codebots.com

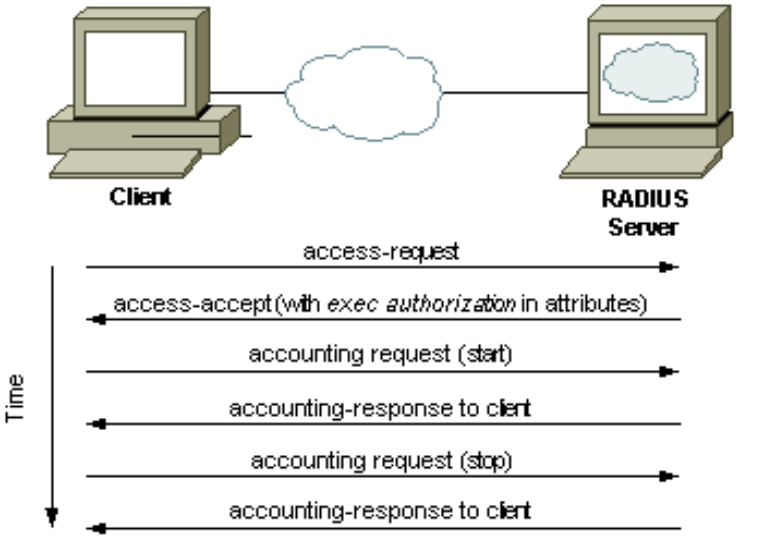
Access management

- It is normally necessary (either for internal control or to comply with legal obligations) to record the accesses granted and denied to entities
- This record is called ***Accounting*** and, together with the previous ***Authentication*** and ***Authorization***, constitutes the **AAA** systems
  - ***AAA - Authentication, Authorization, Accounting***

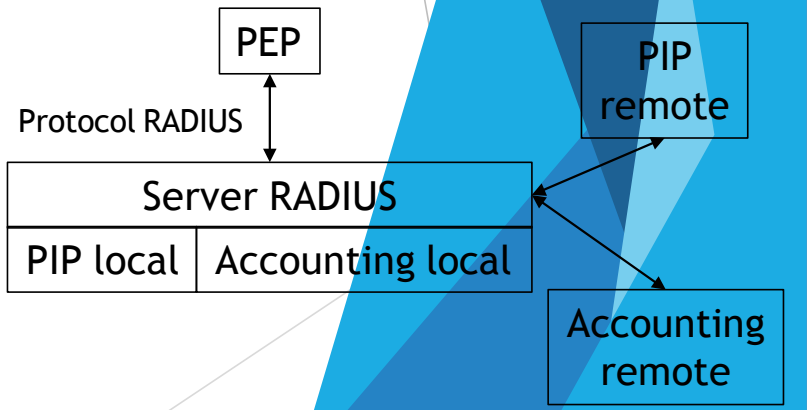


Access management

- Protocols have been developed to ensure support for AAA
- Among these, the ***Remote Authentication Dial In User Service*** (**RADIUS**) (**RFC 2865**)
- RADIUS is both a service and a protocol that ensures communications between the PEP and the PDP
- This communication takes place over UDP to avoid constraints associated with TCP
- It can have a PIP and local accounting or, through extensions, communicate with other services that implement these functions
- In RADIUS, accounting requests are registered only for the beginning and end of the session
- Use 8 bits to set Attribute – Value pairs (AV)

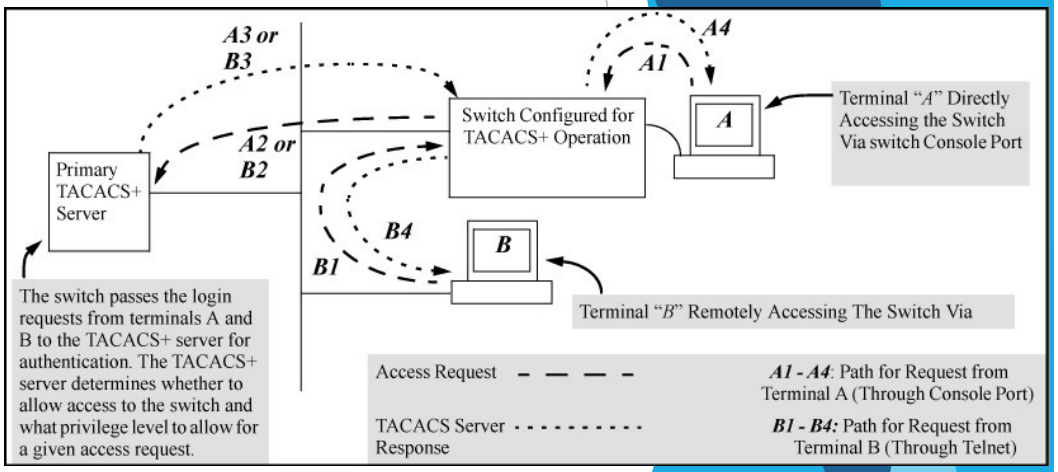


Source: cisco.com



Access management

- Another protocol that supports AAA is the ***Terminal Access Controller Access-Control System Plus*** (**TACACS+**)
  - Evolution of TACACS and XTACACS but incompatible with them
- Unlike RADIUS it uses TCP
- One significant difference is that TACACS+ isolates the checks (authentication and authorization) and repeats the authorization whenever the execution of a new command is required



Source: techhub.hpe.com

Access management

- More recently, in 2003, an evolution based on RADIUS, although not compatible with it, emerged: ***Diameter*** (RFC 6733 in current version)
- As most important aspects,
  - Supports EAP
  - Supports **SCTP** (***Stream Control Transmission Protocol***)
  - Allows *failover* mechanisms
  - It is based on TCP
  - Uses 32 bits to define Attribute-Value pairs (***Attribute-Value Pair*** **AVP**) and is therefore more scalable
    - AVPs contain authentication, accounting, authorization, routing, security and configuration information for the request and response
  - Supports wired networks, Wi-Fi, 3G, IP Multimedia Systems (IMS), and LTE/4G

Access management

- One method used to support AAA is to have a third party ensure identification and authentication, providing *tickets* to entities that request them.
  - Offering what is designated as ***ticket authentication***
- The most common system of this type is ***Kerberos***

Access management

- **Kerberos** was developed by MIT for internal use and later made available for external use
- The latest version is V5, described in RFC 4120
- It offers a ***Single Sign-On*** (**SSO**) solution to users and provides protection for authentication credentials
- It is designed to ensure strong authentication for client/server applications through the use of secret key cryptography
- Through extensions described in RFC 4556, initial authentication can use the ***Public Key Infrastructure*** (**PKI**), to which we will return later

Access management

- Each of the entities to which Kerberos can distribute keys is called a ***principal***
- Kerberos divides infrastructure into ***realms***
- Each *realm* must contain the key distribution service called ***Key Distribution Center*** (**KDC**)
- All users and resources (*principals*) are registered on the KDC, which maintains a database of everyone's keys
- The authentication server performs the functions of the KDC, two distinct and parallel services
  - ***Authentication Service*** (**AS**)
  - ***Ticket Granting Service*** (**TGS**)

Access management

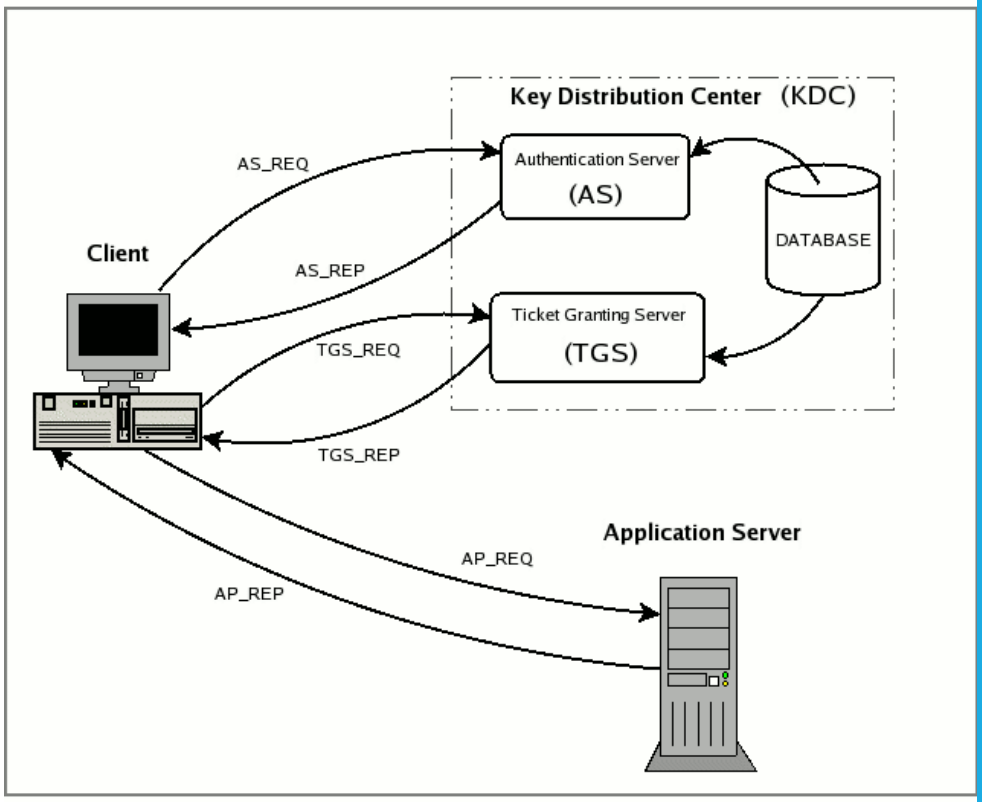
- ***Authentication Server*** (**AS**)
  - Have a key shared with the *principals*
  - Accept or reject the access attempt
  - Provide *principals* with a ticket allowing them to prove their identity to TGS
- ***Ticket Granting Service*** (**TGS**)
  - Provides ***Ticket-Granting tickets*** (**TGT**), session keys for temporary communication, to *principals*
  - Temporary, as TGT has a lifetime
  - If there is no time synchronization on the *principals*, access will be denied
  - TGT proves that the *principal* has authenticated with the KDC and is authorized to request access to resources

Access management

- *Tickets* are encrypted messages that prove authorization to access a resource
- A user asks for a *ticket* to access a resource, and if both (user and resource) are authenticated and the user has permission to use the resource, Kerberos (more exactly TGS) provides it
- In addition to lifetime, Kerberos *tickets* have specific usage parameters
- When a *ticket* expires the customer must request its renewal or a new *ticket* to continue using the feature
- Since all principals are authenticated, it ensures the identity and authorization of each other to the entire structure

Access management

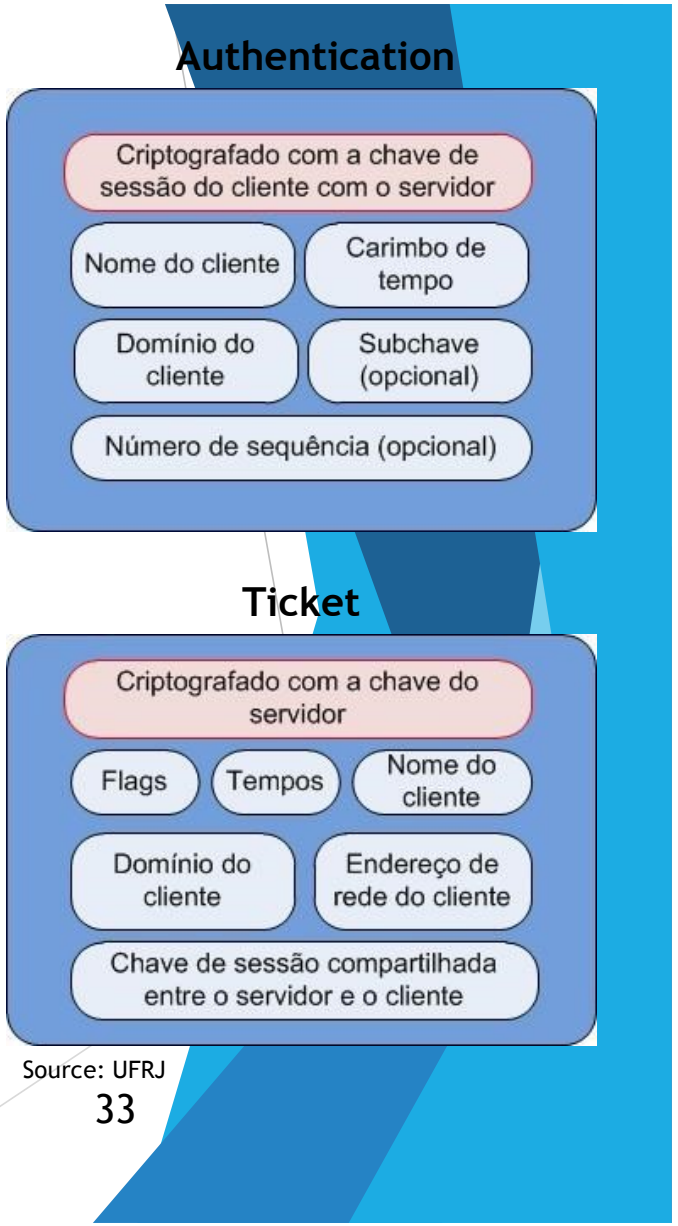
- When a *principal* connects to the *realm* it sends an authentication request to the KDC
- If valid, a Ticket-Granting Ticket (TGT) is returned
- When the *principal* accesses after authenticating a resource on the network, the TGT he received is used to request another ticket to access the resource
- This new request is sent to the KDC which validates it



Source: kerberos.org

Access management

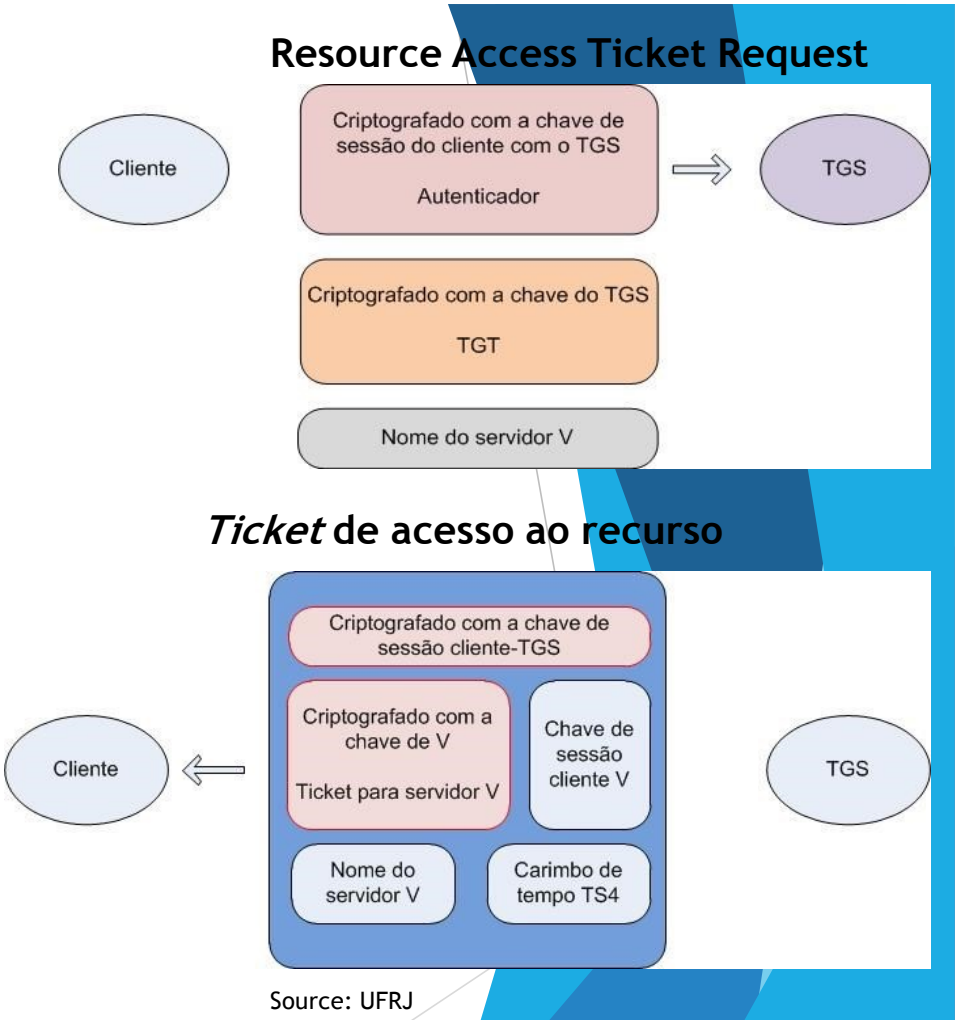
- The Kerberos login process works as follows
  1. User enters authentication on client
  2. The client encrypts the authentication and sends it to the KDC
  3. KDC checks authentication against AS database
  4. The KDC generates a symmetric key to be used in communication; encrypt it with the user's password *hash*; also generates an encrypted TGT
  5. The KDC transmits the symmetric key and the TGT (both encrypted) to the client
  6. The client uses TGT until it expires and decrypts the symmetric key with its *hash*



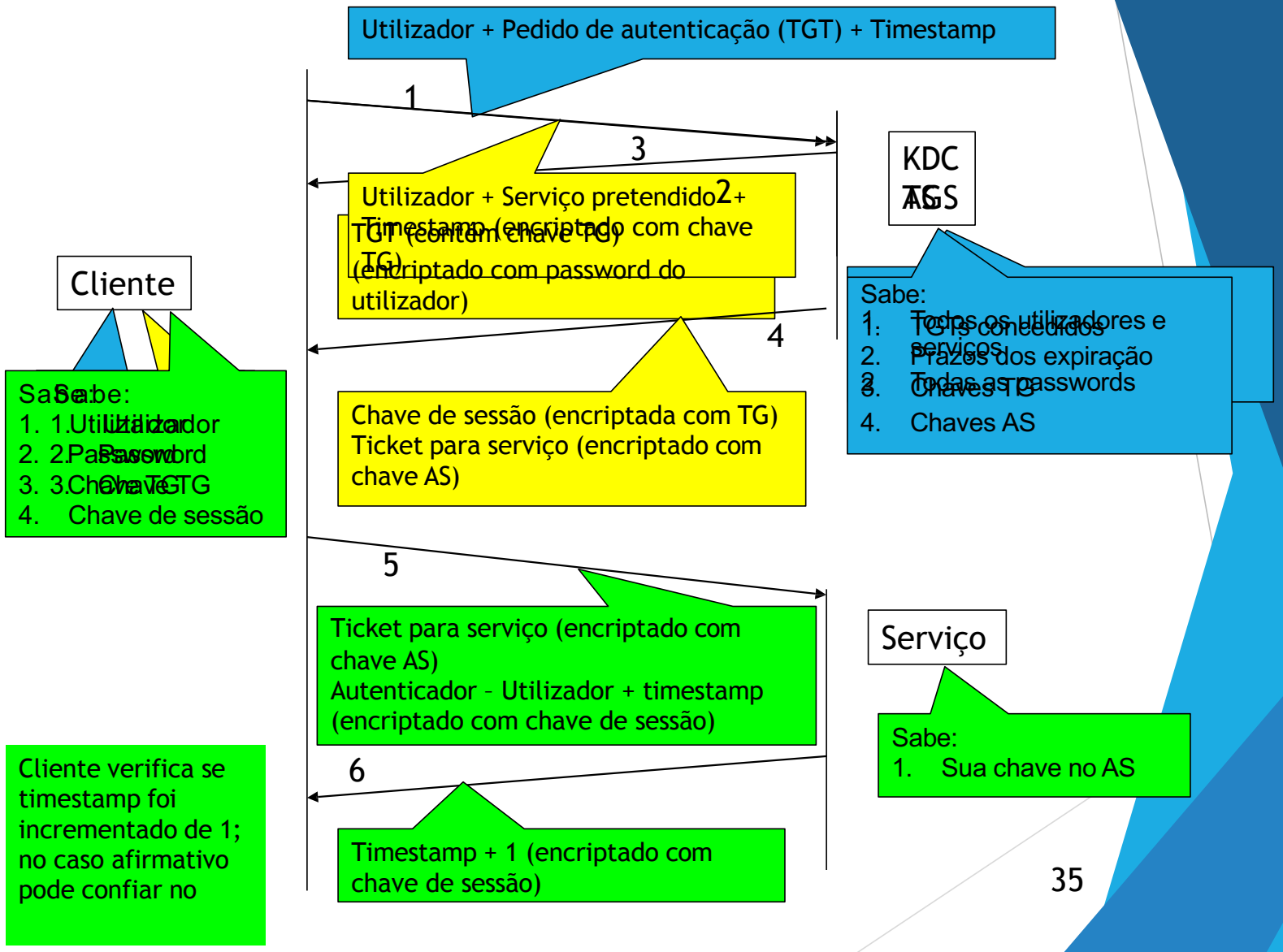
Source: UFRJ

Access management

- When a client wants to access a resource, the sequence is
  1. The client sends his TGT to the KDC along with the resource access request
  2. The KDC checks the validity of the TGT and whether the client is authorized to use this feature
  3. KDC generates a resource access ticket and sends it to the client
  4. The client sends the resource access ticket to the server or service where the resource is hosted
  5. The server or service checks the validity of the ticket with the KDC
  6. After identity verification and access authorization, Kerberos activity is complete
  7. The server or service hosting the resource opens a session with the client and communication is established



Source: UFRJ


Utilizador + Pedido de autenticação (TGT) + Timestamp
1
3
2
Utilizador + Serviço pretendido + Timestamp (encriptado com chave TG)
TGT (contém chave TG) (encriptado com password do utilizador)
KDC
TGS
AS
Cliente
Sabe:
1. Utilizador
2. Password
3. Chave TGT
4. Chave de sessão
Sabe:
1. Todos os utilizadores e serviços
2. Prazos dos expiração
3. Todas as passwords
4. Chaves AS
Chave de sessão (encriptada com TG)
Ticket para serviço (encriptado com chave AS)
5
Ticket para serviço (encriptado com chave AS)
Autenticador – Utilizador + timestamp (encriptado com chave de sessão)
Serviço
Sabe:
1. Sua chave no AS
Cliente verifica se timestamp foi incrementado de 1; no caso afirmativo pode confiar no
6
Timestamp + 1 (encriptado com chave de sessão)

Access management

- Names of *principals* in a realm have reservations
  - The name ***krbgt***, for example, is reserved
  - Symbols **@** and **/** are used to separate the *principal* name from the *realm* name and components of the *principal* name respectively
  - Examples are **user@realm** and **sftp/server@realm**

Access management

- Kerberos can manage trust relationships between different *realms*
- With this trust relationship, it is possible for a *principal* of a *realm* to access resources from a different *realm* without needing to have credentials in the AS of that other *realm*
- This process consists of defining each TGS as a *principal* in the other *realm*
- Regardless of the *realm* they are in, each *principal* can get a ticket to access resources from the other *realm*
- A *realm* is considered to be able to communicate with another *realm* if they both share an *inter-realm key*
  - This key will be used to encrypt remote *realm* resource access request *tickets*

Access management

- Kerberos also supports hierarchy between *realms*
  - An example is Microsoft's Active Directory domains in the same *tree*
- If there is a hierarchy, the trust relationship is transitive, that is, if *realm* A trusts *realm* B and *realm* B trusts *realm* C, then *realm* A trusts *realm* C

Access management

- As a disadvantage of Kerberos, note that KDC is a SPOF
- The keys are based on user passwords, so a "weak" password enhances their capture and subsequent access to the key
- Kerberos does not provide fault tolerance mechanisms, so it is reasonable to configure more than one KDC in the same *realm*, however:
  - The synchronization of databases must be done using external systems (automatic or manual)
  - Applications should be prepared on a trial-error basis, ie if the main KDC does not respond, try the alternate KDC and so on until success or failure after traversing all KDCs
    - This functionality is however available in Microsoft's Active Directory
- If the main KDC is down, Kerberos administration cannot be done until it is restored